Day 12

Hybrid search: BM25 + vectors

Day 11 ended with **"What does Section 103 say?"** and vector search never found it. Today: why **vectors alone are not enough**, what **BM25** does better, and how **hybrid search** joins the two.

1 Where vectors fail | 2 BM25 | 3 Where BM25 fails | 4 Hybrid + RRF | 5 On Kanoon.ai | 6 Recap

1 · where vector search fails

Your RAG is failing silently

Ask for one invoice

query: INV-2023-091	score
1 · INV-2023-**019** · 3 printers · ₹54,000	0.420
2 · INV-2023-**091** · 12 laptops · ₹9,40,000	0.414
3 · INV-2023-910 · 40 chairs	0.371

Wrong invoice first. The LLM answers with ₹54,000. **Nobody notices.**

Ask for one error code

query: error E1042	score
1 · E**1024** · card declined	0.458
2 · E**1042** · gateway timeout	0.448

The bot says "use another card" for a timeout.

why: to the model `INV-2023-091` and `INV-2023-019` are **0.93 alike**. It sees "an invoice number", not **which** one. And it never says "no match": it always returns the nearest thing.

1 · the root cause

Similar meaning ≠ the exact thing

vector search is good at	vector search is bad at
other words: "killing someone" ≈ "causes death"	IDs and codes: `INV-2023-091`, `E1042`, SKUs
the user's own phrasing, typos	numbers: `Section 103`, "Type 1" vs "Type 2"
"close enough"	"exactly this one"

```
search("What does Section 103 say?") → bns::5, bns::3, bns::6 … (Day 11: never bns::61)
```

Embeddings optimise for **semantic similarity**. When the query has a hard detail (an ID, a number, a name), similar is **wrong**. In a demo it looks fine. In production it's the wrong invoice, the wrong patient, the wrong section.

2 · BM25

Keyword search, with a score

No model, no embeddings. Split into words, then count. The classic ranking of search engines (default in Elasticsearch).

TF
word is in the doc, more = better (with a cap)

IDF
rare words count more

length
short doc with the word beats a long one

```
tokenize("Show me invoice INV-2023-091")
→ ['show', 'me', 'invoice', 'inv-2023-091']

idf  invoice       0.45   # in 3 of 8 docs
     inv-2023-091  1.61   # in 1 doc: 3.5× more weight
```

Same two queries

```
bm25("INV-2023-091") → INV-2023-091 1.57
(nothing else)

bm25("error E1042") → E1042 1.73
(nothing else)
```

- **exact match**, and it can say **"no match"**
- fast, local, free, explainable
- the **tokenizer** decides what can match: keep the hyphen, so `INV-2023-091` stays one word

3 · where BM25 fails

Same intent, zero shared words

"cheap family car"

```
doc: "Budget SUVs with 5 seats and a big boot:
Tata Punch and Maruti Brezza, under ₹10 lakh"

bm25 → [] no cheap, family, car
vector → Budget SUVs 0.414 · luxury sedans 0.269
```

Also misses

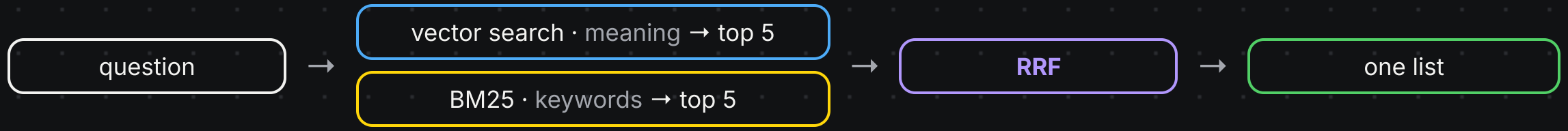
- synonyms: *cheap* vs *budget*
- other forms: *car* vs *cars*
- typos: *invoce*
- other languages: *hatya ki saza* vs *murder*

	finds meaning	finds exact words
vector search	✓	✗
BM25	✗	✓

so: each fails where the other is good. **Use both.**

4 · hybrid search

Two searches, one list: RRF



Can't add the scores: cosine was 0.1–0.5, BM25 has **no fixed range**.
So use only the **rank**:

$$\text{score} = 1/(60 + \text{rank}_1) + 1/(60 + \text{rank}_2)$$

- near the top counts most
- in **both** lists → two shares, it rises
- `60`: from the paper (Cormack et al., 2009)

INV-2023-091, worked out

doc	vector	bm25	RRF
INV-2023-091	2	1	`1/62 + 1/61 = 0.0325`
INV-2023-019	1	-	`1/61 = 0.0164`
INV-2023-910	3	-	`1/63 = 0.0159`

Being in both lists beats being 1st in one.

4 · does it work?

Right on all four

query	vector 1st	BM25 1st	hybrid 1st
`INV-2023-091`	INV-2023-019	INV-2023-091	INV-2023-091
`error E1042`	E1024	E1042	E1042
`cheap family car`	Budget SUVs	(nothing)	Budget SUVs
`how do I change my password`	reset password	reset password	reset password

Benefit

- exact words **and** meaning, one list
- no weights to tune, any score scale
- no need to guess what kind of question it is

Limitation

- two indexes to keep in sync
- ranks only: "a bit better" = "much better"
- a noisy list can push a good doc down

Other way: weighted fusion `α·vector + (1-α)·bm25` after scaling both to 0–1. Tunable, but you **have to** tune it. RRF is the safe default.

5 · on Kanoon.ai

BM25 on the law: mention ≠ section

First try

```
bm25("Section 103")
→ bns::114, ::129, ::125, ::61 …

bns::114: "…punishable under any of
the following sections, namely,
103, 105, 307…"
```

Three chunks that **mention** 103 beat the real Section 103. BM25 counts words; it doesn't know which 103 is a heading.

Fix: index the right words

```
SECTION_START = re.compile(r"\*\*(\d+)\.\*\*")

def bm25_text(chunk):
    sections = " ".join(f"Section {n}" for n in
                        SECTION_START.findall(chunk["text"]))
    return sections + "\n" + chunk["text"]
```

```
bns::61 → "Section 102 Section 103 …"
bm25("Section 103") → bns::61, ::114, ::129 …
```

Only the BM25 text changes. The chunk the user reads and the embeddings stay the same.

5 · the scoreboard

Rank of the right chunk, 198 BNS chunks

question	vector	BM25	hybrid
What does Section 103 say?	-	1	2
What is Section 356?	-	1	2
Section 303 theft	7	1	3
punishment for murder	1	2	1
Is killing someone a crime?	2	-	4
someone posted lies about me on facebook	5	-	9
in the top 5	**3 / 6**	**4 / 6**	**5 / 6**

- = not in the top 10. Top 10 from each search, then RRF.

Read it

- **section numbers:** vector never, BM25 1st, hybrid 2nd–3rd
- **questions in plain words:** vector good, BM25 lost, hybrid still top 5
- **facebook:** BNS never says it; BM25 matched *about*, *on* in unrelated chunks → 5th to **9th**

Not best on every question, rarely bad. Getting the best chunk to 1st is the **reranker**, Day 13.

5 · what hybrid can't fix

Hinglish: no shared word, no help

```
q = "hatya ki saza kya hai"
vector → bns::159, ::73, ::151 … (wrong, Day 11)
bm25 → [] "hatya" is not in the English BNS
hybrid → bns::159, ::73, ::151 … (= vector)
```

Hybrid doesn't fix the vocabulary gap. Each tool fixes one kind of failure, so they stack:

rewrite
Day 11 · Hinglish → law's words

→

hybrid
Day 12 · vector + BM25 + RRF

→

rerank
Day 13 · best chunk 1st

→

context + answer
Day 11 · cite [S1]

in production: the database does the fusion. Qdrant: a sparse and a dense `prefetch`, fused with `RrfQuery(rrf=Rrf(k=60))`.

which one when

What do your users type?

users search for...	use	example
ideas, questions in their own words	**vector**	"is killing someone a crime?"
IDs, codes, SKUs, names, section numbers	**BM25**	`INV-2023-091 · E1042 · Section 103`
both (almost every real app)	**hybrid**	support bots, legal, medical, e-commerce

Vector search

great at **meaning**, weak at **precision**

BM25

great at **precision**, weak at **meaning**

Hybrid

both, **without knowing** which kind of question it is

BEIR (18 datasets): "BM25 is a robust baseline". Keyword search is old, not obsolete.

wrap up

Recap and homework

Recap

meaning	`vector_rank()` · `bns_vector()`
text → words	`tokenize()` `[\w-]+`
exact words	`BM25Okapi` · `bm25_rank()`
index the right words	`bm25_text()`
fuse by rank	`rrf()` `1/(60 + rank)`
both → RRF	`hybrid_rank()` · `bns_hybrid()`

Homework

- add `SKU-7781` and `SKU-7718`: does vector mix them up? hybrid?
- `rrf(k=1)` and `k=200`: how does the invoice table change?
- 5 BNS questions with a section number, 5 without: score all three
- Day 11 `rewrite()` + hybrid on "hatya ki saza kya hai"

Sources, checked 2026-10-31: github.com/dorianbrown/rank_bm25 (rank-bm25 0.2.2) · Cormack, Clarke, Büttcher, SIGIR 2009, cormack.uwaterloo.ca/cormack/cormacksigir09-rrf.pdf · BEIR, arxiv.org/abs/2104.08663 · elastic.co/docs/reference/elasticsearch/index-settings/similarity · qdrant.tech/documentation/search/hybrid-queries · medium.com/@sarthakagg567 "Hybrid search explained"

All numbers from `day-12/code`, run 2026-10-31. Not legal advice.